18. ListIterator — Quick Notes

Definition

- Enhanced version of `Iterator`
- Works only with List collections: ArrayList, LinkedList, Vector

Inheritance:

Iterator
↓
ListIterator

Iterator vs ListIterator

Feature	Iterator	ListIterator
Forward Traversal	✅	✅
Backward Traversal	❌	✅
remove()	✅	✅
set()	❌	✅
add()	❌	✅
hasPrevious()	❌	✅
previous()	❌	✅

Important Methods

Method	Purpose
hasNext()	Check next element
next()	Get next element
hasPrevious()	Check previous element
previous()	Get previous element
add()	Add element while iterating
set()	Replace current element
remove()	Remove current element
nextIndex()	Index of next element
previousIndex()	Index of previous element

Forward Traversal

```
List<Integer> list = Arrays.asList(10,20,30);
ListIterator<Integer> it = list.listIterator();

while(it.hasNext())
  System.out.println(it.next());
```

Output:

```
10
20
30
```

Backward Traversal

Start iterator from end:

```
ListIterator<Integer> it = list.listIterator(list.size());
while(it.hasPrevious())
  System.out.println(it.previous());
```

Output:

```
30
20
10
```

Important

list.listIterator(list.size())
starts iterator at: end of list

---

listIterator(index)

Purpose: Start traversal from any position.

```
ListIterator<Integer> it =list.listIterator(2);
```

Iterator starts at:

```
10 | 20 | 30
       ^
    index 2
```

---

next()

Purpose: Returns next element and moves cursor forward.

Example:

```
it.next();
```

If cursor before:

```
10 | 20 | 30
^
```

Returns:
10
Cursor moves:

```
10 | 20 | 30
  ^
```

---

previous()
Purpose: Returns previous element and moves cursor backward.
Example:

it.previous();
If cursor here:
10 | 20 | 30
          ^

Returns:
30

Cursor moves:
10 | 20 | 30
      ^

---

set()
Purpose: Replace last returned element.

```
List<Integer> list = new ArrayList<>(Arrays.asList(10,20,30));
ListIterator<Integer> it = list.listIterator(list.size());

while(it.hasPrevious()) {
  int x = it.previous();
  it.set(x * 2);
}
```

Output: [20, 40, 60]

Important
`set()` changes element returned by:

- last `next()`
- or last `previous()`

---

add()
Purpose: Insert element during iteration.

```
List<Integer> list = new ArrayList<>(Arrays.asList(10,20,30));
ListIterator<Integer> it =list.listIterator();

while(it.hasNext()) {
  it.add(5);
  System.out.println(it.next());
}
```

Result: [5,10,5,20,5,30]

Important : `add()` inserts at current cursor position.

---

add() Important Rule

After: it.add(5);

- `next()` still points to same element
- `previous()` points to newly added element

---

nextIndex()

Purpose: Returns index of element that would be returned by `next()`.

Example: it.nextIndex();

---

previousIndex()

Purpose: Returns index of element that would be returned by `previous()`.

Example: it.previousIndex();

Example

ListIterator<Integer> it = list.listIterator(2);
For list: [10,20,30]

Cursor position:
10 | 20 | 30
^

Output:
System.out.println(it.previousIndex());
1

Output:
System.out.println(it.nextIndex());
2

Important Quick Points

Topic	Point
ListIterator	Enhanced Iterator
Works With	List collections only
next()	Forward traversal
previous()	Backward traversal
set()	Replace current element
add()	Insert during traversal
nextIndex()	Index of next element
previousIndex()	Index of previous element
listIterator(size())	Start from end
Major Advantage	Bi-directional traversal